

Worksheet No. - 03

Student Name: Sukhleen Kaur

Branch: MCA

Semester: 2nd

Subject Name: Artificial Intelligence Tools

UID: 25MCA20031

Section/Group: 25MCA-6B

Date of Performance: 13.02.2026

Subject Code: 25CAP-657

Aim/Overview of the practical: Implement Best-First Search Algorithm with a chosen heuristic.

Objective: To implement the Best-First Search algorithm, which uses a chosen heuristic to explore the most promising nodes first, in order to efficiently find a path from a start node to a goal node in a graph.

Procedure/Algorithm/Code

Algorithm

1. Initialize the priority queue with the start node.
2. While the queue is not empty, pop the node with the lowest heuristic.
3. If it's the goal, stop and reconstruct the path.
4. Otherwise, add unvisited neighbors to the queue with their heuristic.
5. Repeat until the goal is found or no path exists.

Code

```
import heapq

def best_first_search(graph, heuristic, start, goal):

    priority_queue = []

    visited = set()

    came_from = {start: None}
```

```
# Push start node into priority queue with its heuristic
```

```
heapq.heappush(priority_queue, (heuristic[start], start))
```

```
while priority_queue:
```

```
    current_h, current_node = heapq.heappop(priority_queue)
```

```
    print(f"Visiting: {current_node} with Heuristic: {current_h}")
```

```
    if current_node == goal:
```

```
        print("Goal Reached")
```

```
    path = []
```

```
    while current_node:
```

```
        path.append(current_node)
```

```
        current_node = came_from[current_node]
```

```
    path.reverse()
```

```
    print("Path:", ' -> '.join(path))
```

```
    return path
```

```
visited.add(current_node)
```

```
for neighbor in graph.get(current_node, []):
```

```
    if neighbor not in visited and neighbor not in [n for _, n in priority_queue]:
```

```
        came_from[neighbor] = current_node
```

```
        heapq.heappush(priority_queue, (heuristic[neighbor], neighbor))
```

```
print("No path found from", start, "to", goal)

return None

# Graph definition

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['G'],
    'F': [],
    'G': []
}

# Heuristic values

heuristic = {
    'A': 6,
    'B': 4,
    'C': 5,
    'D': 7,
    'E': 2,
    'F': 8,
    'G': 0
}

start_node = input("Enter the Start Node: ").strip().upper()
goal_node = input("Enter the Goal Node: ").strip().upper()
```

if start_node not in graph or goal_node not in graph:

```
print("Invalid Start or Goal Node.")
```

else:

```
best_first_search(graph, heuristic, start_node, goal_node)
```

Output

```
Enter the Start Node: A
Enter the Goal Node: D
Visiting: A with Heuristic: 6
Visiting: B with Heuristic: 4
Visiting: E with Heuristic: 2
Visiting: G with Heuristic: 0
Visiting: C with Heuristic: 5
Visiting: D with Heuristic: 7
Goal Reached
Path: A -> B -> D
```

Learning outcomes (What I have learnt):

1. Understood the concept of Best-First Search and how it differs from other search algorithms.
2. Learnt how to use a heuristic function to guide the search efficiently.
3. Gained experience in graph traversal and pathfinding techniques.
4. Learnt to use priority queues to select the most promising nodes.